

COMPILAÇÃO E TIPOS PRIMITIVOS

LINGUAGEM DE PROGRAMAÇÃO

TALITA BERBEL

talita.berbel@fatec.sp.gov.br

Fatec
Sorocaba
José Crespo Gonzales

CP
Centro
Paula Souza

AGENDA

- Processo de compilação
- Opções de editores e ambiente de compilação
- Como estudar um programa de computador
- Tipos de dados primitivos
- Estudos e codificação
- Dúvidas

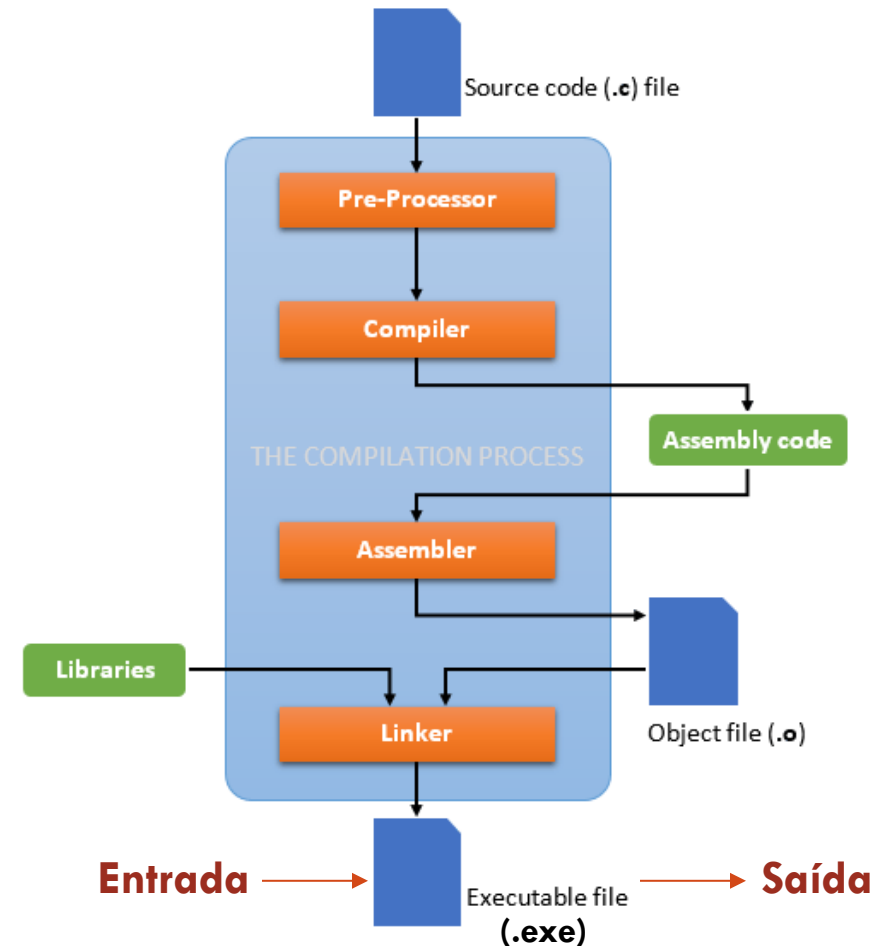


PROCESSO DE COMPILAÇÃO

- O que é o gcc?
- gcc significa GNU Compiler Collection e foi inicialmente escrito para ser o compilador do sistema operacional GNU. Ele contém compiladores para as seguintes linguagens: C, C++, Java e outras.
- O que é main.c?
- main.c é o arquivo que será compilado. Esse arquivo é escrito na linguagem de programação C. Os programadores também o chamam de código-fonte, que é uma lista de instruções legíveis por humanos escritas por um programador. A nomeação do arquivo fica a critério do programador, ou seja, não precisa ser necessariamente “main”, mas é necessário ter um função main() dentro desse arquivo.

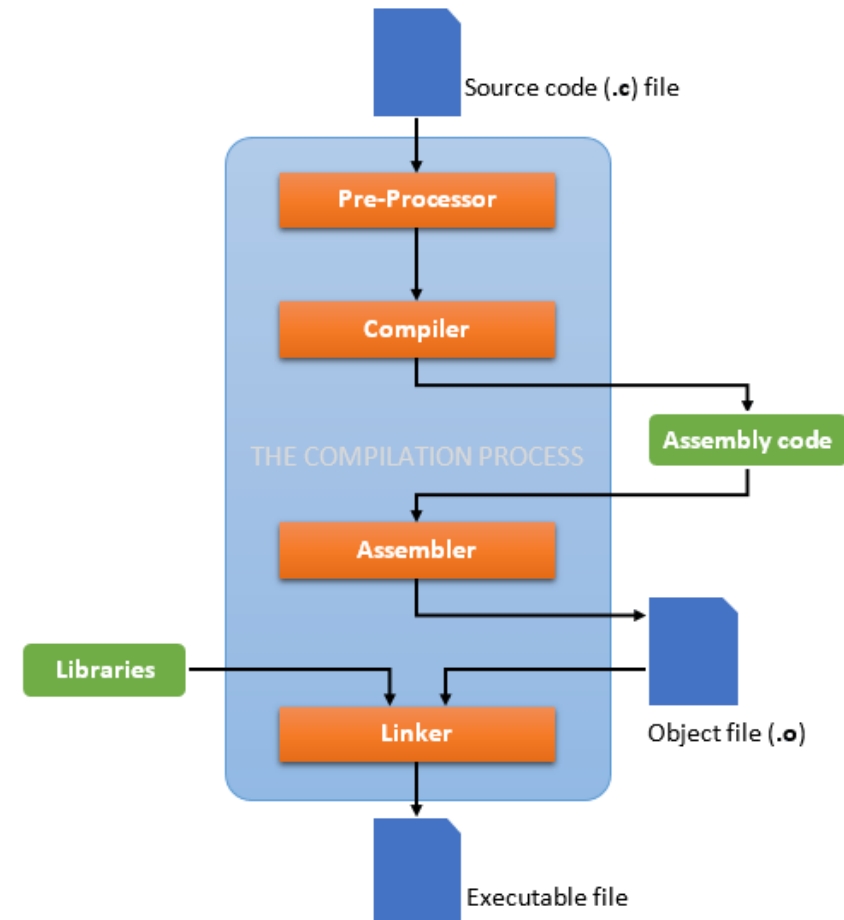
PROCESSO DE COMPILAÇÃO

- O processo de compilação é dividido em quatro partes:
- **Pré-processador,**
- **Compilador,**
- **Montador (Assembler) e**
- **Ligador (Linker).**



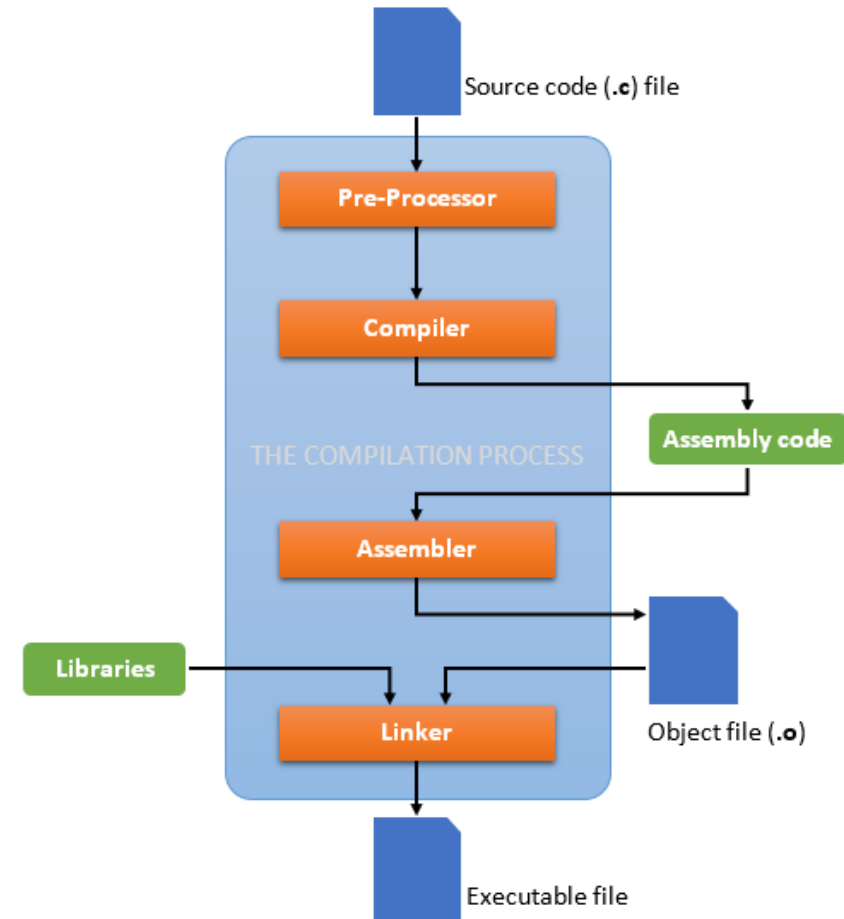
PROCESSO DE COMPILAÇÃO

- O processo de compilação é dividido em quatro partes:
- **Pré-processador** - O código-fonte passa pelo pré-processador, que realiza três tarefas:
 - Remove os comentários;
 - Inclui os arquivos de cabeçalho (header files);
 - Substitui nomes de macros pelo respectivo código.
- **Compilador** - Depois do pré-processamento, o compilador converte tudo em código de montagem (assembly), que é uma combinação de texto simples e código legível por humanos.



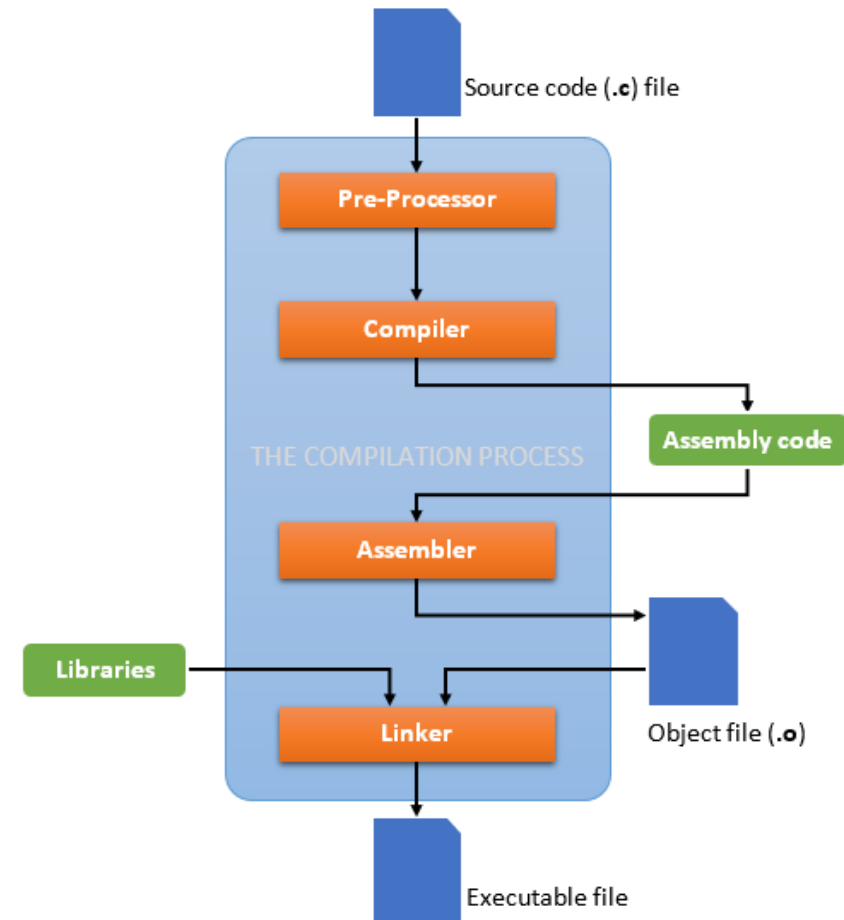
PROCESSO DE COMPILAÇÃO

- O processo de compilação é dividido em quatro partes:
- **Montador (Assembler)** - O código assembly é então enviado ao montador, que o converte em código objeto.
- O código objeto é composto por binários (0 e 1) e também é chamado de linguagem de máquina.
- **Ligador (Linker)** - Por fim, o ligador conecta o código objeto com bibliotecas compartilhadas ou dinâmicas.



PROCESSO DE COMPILAÇÃO

- O processo de compilação é dividido em quatro partes na **cadeia gcc**:
- **Pré-processador** → resolve macros e inclui cabeçalhos.
- **Compilador** → converte para em código de montagem (assembly).
- **Montador** (*Assembler*) → converte assembly para código objeto (binário).
- **Ligador** (*Linker*) → une tudo e produz o executável.



OPÇÕES DE EDITORES E AMBIENTES DE COMPILAÇÃO

- **VS Code** – Editor de código-fonte leve e altamente extensível, com suporte a múltiplas linguagens via extensões (incluindo C/C++).
- **Visual Studio** – IDE completa da Microsoft, com recursos avançados de depuração, design de interface e integração para várias linguagens.
- **Dev-C++** – IDE voltada para programação em C e C++, simples e leve, geralmente usada com o compilador MinGW.
- **Code::Blocks** – IDE gratuita e modular para C, C++ e Fortran, compatível com diferentes compiladores.
- **OnlineGDB** – IDE e depurador online que permite compilar e executar código em C/C++ (e outras linguagens) diretamente no navegador.
- **Notepad++ e Cygwin** – Notepad++ é um editor de texto com suporte a sintaxe; Cygwin fornece um ambiente Unix/Linux dentro do Windows, permitindo compilar e rodar programas C/C++ pelo terminal.

COMO ESTUDAR UM PROGRAMA DE COMPUTADOR

1. Comentar o **código impresso** (a lápis), explicando cada linha do programa.
2. Tentar resolver dúvidas **sozinho** primeiro, pesquisando em livros, apostilas, notas de aula e internet.
3. **Perguntar** ao monitor, colega ou professor somente após a pesquisa individual.
4. Monitor pode revisar os comentários do aluno e sugerir correções.
5. Realizar **teste de mesa em papel**, simulando a execução e anotando o resultado final.
6. Novamente tentar resolver dúvidas **sozinho** na simulação, antes de pedir ajuda.
7. Codificar, compilar e executar o programa no ambiente de desenvolvimento.
8. Executar com os mesmos valores do teste de mesa e comparar os resultados.
9. Identificar e entender erros encontrados na comparação.
10. Fazer pequenas modificações no código para ampliar o domínio da linguagem.



TIPOS DE DADOS PRIMITIVOS

- Acessar PDF no Teams sobre Tipos de Dados Primitivos.



TIPOS DE DADOS PRIMITIVOS

- Os **tipos primitivos** são tipos básicos, pré-definidos no compilador, também chamados de tipos *embutidos* ou *nativos*.
- Categorias principais:
 - Numéricos inteiros (com e sem sinal, diferentes tamanhos e faixas).
 - Numéricos reais (com parte fracionária, diferentes precisões).
- Diferença entre C e Pascal: nomes, tamanhos e faixas de valores variam conforme a linguagem e plataforma.
- Importância: base para criação de tipos mais complexos ou compostos, chamados de *tipos construídos*, tais como, struct, union, enum e arrays.

TIPOS DE DADOS PRIMITIVOS

- Os **tipos primitivos** são tipos básicos, pré-definidos no compilador, também chamados de tipos *embutidos* ou *nativos*.
- Categorias principais:
 - Numéricos inteiros (com e sem sinal, diferentes tamanhos e faixas).
 - Numéricos reais (com parte fracionária, diferentes precisões).
- Diferença entre C e Pascal: nomes, tamanhos e faixas de valores variam conforme a linguagem e plataforma.
- Importância: base para criação de tipos mais complexos ou compostos, chamados de *tipos construídos*, tais como, struct, union, enum e arrays.

TIPOS DE DADOS PRIMITIVOS

- **Contagem com unsigned char e int**
- O que faz: incrementa variáveis cNum (unsigned char) e iNum (int) até um limite.
- Relação com tipos primitivos: mostra limite de faixa do unsigned char (0 a 255) e o comportamento de overflow.

```
#include <stdio.h>

int main(void)
{
    unsigned char cNum;
    int iNum;

    cNum = 0;
    iNum = 0;
    while (cNum <= 260)
    {
        printf("%d %d\n", iNum, cNum);
        iNum = iNum + 1;
        cNum = cNum + 1;
    }

    return 0;
}
```

TIPOS DE DADOS PRIMITIVOS

- **Subtração com float**
- O que faz: subtrai 0.1 repetidamente de um valor inicial até atingir um número de iterações.
- Relação com tipos primitivos: evidencia perda de precisão em tipos reais (float) devido à representação binária.

```
#include <stdio.h>

int main(void)
{   float valor = 1000.0;
    int cont = 0;

    while (cont < 1000 * 10)
    {   valor = valor - 0.1;
        cont = cont + 1;
    }

    printf("\nValor: %f  %d",
           valor, cont);

    return 0;
}
```

TIPOS DE DADOS PRIMITIVOS

- **Uso do sizeof**
- O que faz: exibe o tamanho (em bytes) de diferentes tipos (char, short int, long int, int, float, double).
- Relação com tipos primitivos: demonstra quantidade de memória que cada tipo ocupa, dependente da plataforma e compilador.

```
#include <stdio.h>

int main(void)
{
    char v1;
    short int v2;
    long int v3;
    int v4;
    float v5;

    printf("%d %d %d %d %d %d\n",
           sizeof(v1), sizeof(v2),
           sizeof(v3), sizeof(v4),
           sizeof(v5), sizeof(double));

    return 0;
}
```

TIPOS DE DADOS PRIMITIVOS

- **Divisão inteira atribuída a float**
- O que faz: atribui o resultado de $5 / 2$ a uma variável float.
- Relação com tipos primitivos: mostra que operadores dependem do tipo dos operandos, ou seja, a divisão entre inteiros descarta parte fracionária, mesmo que o resultado seja guardado em um float. Existem alternativas para resolver esse problema.

```
#include <stdio.h>

int main(void)
{   float valor;

    valor = 5 / 2;

    printf("%f\n", valor);

    return 0;
}
```

DÚVIDAS

- Questionamentos?

